

Srinivasan Raghavan

Boston, MA | r.srini5303@gmail.com | LinkedIn | GitHub | +1 (857) 351-9661

EDUCATION

- Northeastern University**

M.S. in Artificial Intelligence; GPA : 3.75/4.0

Boston, MA

September 2024 - April 2026

SKILLS

- **Agentic & Orchestration:** LangGraph, Multi-Agent Systems, Zypher, Agent Orchestration, RAG, Prompt Engineering
- **Programming Languages:** Python, R, Java, SQL
- **Libraries & Frameworks:** NumPy, Pandas, TensorFlow, PyTorch, Keras, scikit-learn, SciPy, LangChain, NLTK
- **Machine Learning & GenAI:** Statistical Modeling, Data Analysis, Transformers, LLMs, NLP, PEFT
- **Databases & Visualization:** MySQL, PostgreSQL, MongoDB, Pinecone, ChromaDB, Matplotlib, Power BI
- **Tools & Platforms:** FastAPI, Git, Docker, Kubernetes, AWS EC2, GCP, Azure, vLLM, Hugging Face, OpenAI, BigQuery, MATLAB

EXPERIENCE

- AI Intern | Databricks, Agent orchestration, RAG**

Brightstar Lottery

Providence, RI

June 2026 - Present

- Engineered a Databricks ingestion pipeline (Lakeflow Connect) parsing multi-format SharePoint documents into Unity Catalog tables, applying deterministic chunk-level primary keys to enable reliable downstream retrieval for RAG workflows.
- Deployed retrieval infrastructure on Databricks Vector Search to power a production RAG-based Knowledge Assistant agent, delivering source-cited, grounded answers to 100+ finance users.
- Built a model routing mechanism using the Databricks Python SDK alongside a grounding model logged and registered in MLflow, integrating both into agent orchestration to improve reliability and cut inference cost by 35%.

- Machine Learning Intern | DSL, NLP Evaluation, vLLM**

TechConative

Chennai, India

October 2023 - April 2024

- Engineered a streamlined evaluation pipeline for DSL generation using LoRA-finetuned LLaMA model, reducing evaluation time by 65% and implementing comprehensive NLP metrics (accuracy, ROUGE, BLEU) to quantify code generation quality.
- Deployed a production vLLM inference server with PEFT adapters, leveraging paged attention mechanism for efficient memory management that achieved 24x throughput improvement over baseline Hugging Face Transformers implementation.

PROJECTS

- DeadPool | Python, PyTorch, FAISS, FastAPI, MLflow, LangGraph**

- Architected a multi-agent orchestration framework using LangGraph with 6 modular, independently extensible specialist agents across multiple departments, coordinated by a Head Agent that synthesizes cross-domain signals into a unified risk score.
- Built a FastAPI service layer with SSE streaming for sub-second updates, a Pydantic v2 schema contract for type-safe agent communication, and a cascade expansion engine mapping failure propagation across domains, enabling early detection of compounding failures.
- Developed a React 19 frontend with React Flow for interactive cascade visualization, integrating Slack and GitHub APIs for live signal ingestion across 2+ external data sources, deployed on Railway with a Caddy reverse proxy for production-grade serving.

- AgentPlanner | FastAPI, Multi-Agent, RAG, LLMs, GCP**

- Architected an intelligent project decomposition system using LangGraph-orchestrated multi-agent workflows that convert problem statements into atomic tasks with complexity, timelines, dependencies, and priority, reducing design planning time by 90%.
- Assembled and integrated 4 specialized agents (Planner, Timeline, Dependency, Formatter) with RAG on a custom dataset of 500+ historical projects, achieving 87% accuracy in task breakdown, scheduling, and structured JSON outputs.
- Built a React-based interface connected to LLM agents via a FastAPI backend, deployed on Google Cloud Run with intelligent caching for a 40% cost reduction.

- EchoAI | Python, React, Google Cloud Run, GitHub Actions**

- Automated restaurant review management reducing response time from 2-3 days to under 1 minute, enabling restaurants to maintain 95%+ review response rates and improve customer satisfaction scores by addressing negative feedback 48x faster.
- Designed and maintained an end-to-end MLOps stack (Apify→Firestore→Airflow→HF LLMs) using Docker with data validation pipelines, MLflow for experiment tracking with 10+ models and GCR model registry to ensure reproducibility and reliable deployments.
- Established CI/CD with GitHub Actions and orchestrated multi-model workflows using Apache Airflow (staged testing, automated rollback, environment promotion, the same discipline used in Jenkins/GitLab Runner pipelines), enabling human-free retraining and scalable model serving on Google Cloud Run with <5 min of deployment time.